

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: ARTIFICIAL INTELLIGENCE

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

Duration: 30 Minutes

Questions: 20

Total Marks: 20

Annexure A

MCQ Test

Code of Conduct:

I S. Karthikeyan / 192419048 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

CSA17 — ARTIFICIAL INTELLIGENCE

Assessment Tool 1 — MCQ Test: Answer Key with Explanations

CO2: Search and game-solving techniques — Greedy Search, A*, CSP, Minimax, Alpha-Beta pruning, heuristic functions

Q1. Which algorithm selects the node to expand next using only the heuristic value (estimated closeness to the goal), without considering the cost already incurred?

- A) Breadth First Search
- B) Uniform Cost Search
- C) Greedy Best-First Search ✓ Correct**
- D) Depth First Search

Why: Greedy Best-First Search expands the node with the lowest $h(n)$ — the estimated distance to the goal — completely ignoring $g(n)$, the cost already spent reaching that node. This makes it fast but not guaranteed optimal, unlike A*, which also factors in the accumulated cost.

Q2. What is the evaluation function used in A* search?

- A) $f(n) = g(n)$
- B) $f(n) = h(n)$
- C) $f(n) = g(n) + h(n)$ ✓ Correct**
- D) $f(n) = g(n) \times h(n)$

Why: A* combines the actual cost so far, $g(n)$, with the estimated remaining cost to the goal, $h(n)$, by simple addition: $f(n) = g(n) + h(n)$. This balance of "cost so far" and "cost to go" is exactly what lets A* find optimal paths when $h(n)$ is admissible.

Q3. What property of the heuristic function ensures optimality in A* search?

- A) Completeness
- B) Admissibility ✓ Correct**
- C) Depth limit
- D) Backtracking

Why: A heuristic is admissible if it never overestimates the true cost to reach the goal. As long as $h(n)$ stays admissible, A* is guaranteed to find the optimal (lowest-cost) solution; if it overestimates even once, that guarantee breaks.

Q4. Which of the following is a local search algorithm, used when the path to the solution does not matter and only the final state does?

- A) A* Search
- B) Hill Climbing ✓ Correct**
- C) Breadth First Search
- D) Uniform Cost Search

Why: Hill Climbing starts from one candidate state and repeatedly moves to a better neighbouring state based on an objective function, keeping no memory of the path taken — the defining trait of a local search algorithm, unlike A*/BFS/UCS which build and track an explicit search tree/path.

Q5. Which algorithm is commonly used for adversarial decision-making, where an agent must consider both its own moves and an opponent's possible responses?

- A) Greedy Search
- B) Minimax Algorithm ✓ Correct**

- C) Backtracking
- D) Uniform Search

Why: Minimax explicitly models two competing players: the maximizing agent picks the move that leads to the highest achievable outcome, while assuming the opponent (the minimizer) will always respond with the move worst for the first agent — the core mechanism behind adversarial game-tree search.

Q6. A delivery robot selects paths based only on the estimated distance to the destination, without considering obstacles/costs already encountered, sometimes leading to suboptimal but fast decisions. Which algorithm is this?

- A) A* Search
- B) Greedy Best-First Search ✓ Correct**
- C) Uniform Cost Search
- D) Depth Limited Search

Why: Using only the estimated remaining distance ($h(n)$) and ignoring cost already incurred is exactly Greedy Best-First Search's behaviour — fast because it always chases what looks closest to the goal, but not optimal because it never reconsiders cost already spent.

Q7. A GPS system calculates routes by combining the distance already travelled with an estimate of the remaining distance to the destination. Which algorithm best represents this?

- A) Depth First Search
- B) A* Search ✓ Correct**
- C) Hill Climbing
- D) Backtracking

Why: Combining "distance already travelled" ($g(n)$) with "an estimate of what remains" ($h(n)$) is precisely A's evaluation function $f(n) = g(n) + h(n)$, which is why A* is the standard algorithm behind real shortest-route GPS navigation.*

Q8. A university must schedule exams for multiple subjects with constraints such as no overlapping exams and limited rooms. This problem is best modeled as:

- A) Game Playing Problem
- B) Constraint Satisfaction Problem ✓ Correct**
- C) Heuristic Search Problem
- D) Optimization Problem

Why: Exam scheduling has variables (each exam), domains (available time slots/rooms), and explicit constraints that must all be satisfied simultaneously (no clashes, room capacity) — the textbook definition of a Constraint Satisfaction Problem (CSP).

Q9. An AI chess program evaluates all possible moves and anticipates the opponent's responses, assigning utility values to game states to determine the best move. Which algorithm is this?

- A) Greedy Search
- B) Hill Climbing
- C) Minimax Algorithm ✓ Correct**
- D) Breadth First Search

Why: Assigning utility values to game states and reasoning through the opponent's best possible counter-moves before deciding is the defining behaviour of the Minimax algorithm used in two-player adversarial games like chess.

Q10. A robot explores an unknown environment with no prior knowledge of obstacles, deciding step-by-step and updating its knowledge as it moves. Which type of agent is this?

- A) Offline Agent

B) Online Search Agent ✓ Correct

C) Static Agent

D) Deterministic Agent

Why: An Online Search Agent interleaves computation and action: it senses, decides one step at a time, and revises its knowledge as new information is discovered — necessary here since the map/obstacles are not known in advance (unlike an offline agent, which would need the full map upfront).

Q11. Why is an admissible heuristic important in A* search?

A) It reduces memory usage

B) It ensures optimality ✓ Correct

C) It increases branching factor

D) It avoids loops

Why: An admissible heuristic never overestimates the true remaining cost, which is exactly the condition A's optimality proof depends on — this is what guarantees the first solution A* finds via the goal is truly the cheapest one, not merely a reduction in memory or branching.*

Q12. In large game trees, evaluating every node is computationally expensive. What is the primary benefit of Alpha-Beta pruning?

A) Reduces depth of tree

B) Reduces number of nodes evaluated ✓ Correct

C) Changes game rules

D) Improves heuristic accuracy

Why: Alpha-Beta pruning cuts off branches that are provably irrelevant to the final minimax decision — it does not shrink the tree's actual depth or alter the game, it simply avoids evaluating nodes that cannot influence the outcome, directly reducing the number of nodes visited.

Q13. Hill Climbing continuously moves toward better states based on local improvements but may fail to reach the global optimum. What is its main drawback?

A) High memory usage

B) Revisiting nodes

C) Getting stuck in local maxima ✓ Correct

D) Requires complete tree

Why: Because Hill Climbing only ever accepts a move if it strictly improves the current state, it can get trapped at a local maximum — a state better than all its immediate neighbours but still worse than the true global optimum — with no mechanism to escape.

Q14. CSPs involve variables, domains, and constraints. What ensures a valid CSP solution?

A) Heuristic function

B) Path cost

C) Constraint satisfaction ✓ Correct

D) Depth-first traversal

Why: By definition, a CSP solution is valid only when every variable is assigned a value from its domain such that all stated constraints hold simultaneously — constraint satisfaction itself is the acceptance test, independent of any heuristic, path cost, or search order used to find it.

Q15. In adversarial games, evaluation functions are used when exploring the entire game tree is infeasible. What is the role of an evaluation function?

A) Generate successors

B) Estimate utility of a state ✓ Correct

C) Reduce branching factor

D) Store moves

Why: An evaluation function assigns an approximate desirability/utility score to a non-terminal game state when the search has to stop before reaching a true end-of-game outcome — letting Minimax/Alpha-Beta make a reasoned decision without expanding the full tree.

Q16. In A* search, a node has a path cost of 7 and a heuristic estimate of 5. What is the value of $f(n)$?

A) 12 ✓ Correct

B) 2

C) 35

D) 7

Why: $f(n) = g(n) + h(n) = 7 + 5 = 12$. (Option B mistakenly subtracts, option C mistakenly multiplies, and option D uses only $g(n)$ and ignores $h(n)$ — none apply A*'s actual additive evaluation function.)

Q17. A heuristic function overestimates the actual cost to reach the goal, violating admissibility. What happens in such a case?

A) A* remains optimal

B) A* becomes non-optimal ✓ Correct

C) A* becomes BFS

D) A* stops working

Why: Admissibility (never overestimating) is precisely the condition A*'s optimality guarantee relies on. Once a heuristic overestimates the true cost, A* can be misled into favouring a more expensive path over a cheaper one, so it may still terminate with a solution — just not necessarily the optimal one.

Q18. A search tree has a branching factor of 2 and a depth of 4. What is the total number of nodes in the full tree?

A) 10

B) 15

C) 31 ✓ Correct

D) 16

Why: Total nodes across all levels 0 through 4 = $2^0 + 2^1 + 2^2 + 2^3 + 2^4 = 1 + 2 + 4 + 8 + 16 = 31$, matching the standard formula $(b^{d+1} - 1)/(b - 1) = (2^5 - 1)/(2 - 1) = 31$. (16 alone would only be the leaf count at depth 4, not the tree's total.)

Q19. In a minimax tree, each node has 3 children and the depth of the tree is 3. How many leaf nodes are present?

A) 9

B) 27 ✓ Correct

C) 6

D) 12

Why: Leaf-node count = branching factor raised to the depth = $b^d = 3^3 = 27$. This matches the standard way minimax game-tree size is estimated from branching factor and depth.

Q20. Alpha-Beta pruning improves the efficiency of minimax by reducing the number of nodes evaluated. What is the best-case time complexity?

A) $O(b^d)$

B) $O(b^{(d/2)})$ ✓ Correct

C) $O(d^b)$

D) $O(\log b)$

Why: In the best case (when moves happen to be ordered so the strongest pruning occurs at every level), Alpha-Beta effectively only needs to examine the equivalent of a tree with half the depth's branching applied twice, giving $O(b^{d/2})$ — a dramatic improvement over plain minimax's $O(b^d)$.

Quick-Reference Answer Key

Q#	Ans	Q#	Ans
1	C	11	B
2	C	12	B
3	B	13	C
4	B	14	C
5	B	15	B
6	B	16	A
7	B	17	B
8	B	18	C
9	C	19	B
10	B	20	B